FLUXBASE

Integration Guide

Building resilient, data-driven backends with raw SQL, Webhooks, and Real-time SSE.

Node.js	Python	Go	Rust
Java	Ruby	PHP	CURL

Version 4.1 • Comprehensive Edition

April 2026

Table of Contents

01. Introduction & Architecture 03

02. Getting Started: Projects & API Keys 05

03. Core SQL Execution API 07

04. Advanced Query Patterns 09

05. Language Tutorial: Node.js (Detailed) 11

06. Language Tutorial: Python (Detailed) 13

07. Language Tutorial: Go Lang (Detailed) 15

08. Language Tutorial: Rust (Detailed) 17

09. Language Tutorial: Java (Detailed) 19

10. Webhooks: Event-Driven Lifecycle 21

11. Storage: S3 Buckets & Uploads 23

12. Real-time: SSE & Sidecar Architecture 25

13. Case Study: Building a Live Chat App 27

14. Organization Suspension & Governance 30

15. Error Codes & Rate Limits 32

© 2026 Fluxbase Inc.

01. Unified Database Architecture

Fluxbase is more than just a storage layer; it is a unified database middleware that exposes a simplified, secure SQL interface over enterprise-grade cloud compute. This guide focuses on integrating your **external backends** with Fluxbase to create a hybrid infrastructure.

The Fluxbase Ecosystem

SQL-First Pipeline: Every operation in Fluxbase—whether through the Dashboard or API—executes compliant SQL. We do not use proprietary JSON query languages. If you know SQL, you know Fluxbase.

DIRECT REST ACCESS

Standardized stateless communication for CRUD operations. Ideal for Lambda functions, serverless workers, and legacy backends.

PERSISTENT SSE

Stateful subscriptions that broadcast database changes instantly across global regions via our Render sidecars.

Why Integrated SQL?

Traditional ORMs often abstract away performance. Fluxbase keeps you close to the data, allowing for complex joins, window functions, and DDL migrations through a single `POST` request.

System Data Flow

```
                        |-------------------|
              |   YOUR BACKEND    | <--- (1) AUTH & API KEY
                        | (Node, Python,etc)|
                        |-------------------|
                                    |
                     | (2) POST /api/execute-sql (SQL Query)
                                    v
            |-------------------|      (3) Execute SQL
 |   FLUXBASE API    | ----------------------> [ AWS RDS / PG ]
 |    (VERCEL)       | <---------------------- [   DATA SET   ]
                        |-------------------|
                                    |
                      | (4) FORWARD EVENTS (INTERNAL BUS)
                                    v
           |-------------------|      (5) OUTBOUND POST
 |  WEBHOOK ENGINE   | ----------------------> [ YOUR APP URL ]
                        |-------------------|
                                    |
                        | (6) EMIT SSE (LIVE NOTIFY)
                                    v
         |-------------------|      (7) PERSISTENT STREAM
|  RENDER SIDECAR   | ----------------------> [ FRONTEND / SDK ]
                        |-------------------|
```

Global Deployment Strategy

- **Primary API:** Hosted on Vercel Edge for lowest latency SQL routing.
- **Storage:** Backed by AWS S3 & CloudFront for worldwide asset delivery.
- **Real-time:** Dedicated Render instances maintain 50,000+ concurrent TCP connections.

02. Projects & Authentication

Before executing your first query, you must provision an organization and generate secure project-level credentials.

01

LOCATING YOUR PROJECT ID

Every project in Fluxbase has a unique identifier (ULID). You can find this in the **Project Settings** panel of your dashboard. This ID is required to route your SQL queries to the correct database schema.

```
project_98b50e2ddc9943efb387052637738f61
```

02

GENERATING API KEYS

Authentication is handled via Bearer Tokens. We recommend creating "Scoped Keys" that are limited to a specific project. This prevents a single compromised key from affecting your entire organization.

Go to: Settings > API Keys > Create New Key

Prefix: All keys begin with `fl_...` or `sk_...`

Security Tip: Never expose your Secret API Key in frontend code. For client-side reactions, use Public Keys with restricted scopes or SSE subscriptions.

The Authorization Header

Requests without a valid header will be rejected with a `401 Unauthorized` status. All communication must occur over TLS 1.3.

```
HTTP
POST /api/execute-sql HTTP/1.1
Host: fluxbase.vercel.app
Content-Type: application/json
Authorization: Bearer fl_live_550e8400e29b41d4a716446655440000
User-Agent: Fluxbase-Integration-Client/1.0
```

Role-Based Access (RBAC)

Your API Key inherits the permissions of the user who created it, or the specific service account it is assigned to. Common roles include:

ROLE	PERMISSIONS
Admin	Full DDL (Create/Drop Tables) and DML access.
Developer	Table mutations and data queries. No database deletions.
Viewer	Read-only access (SELECT only).

03. SQL Execution Pipeline

The `/api/execute-sql` endpoint is the heart of the Fluxbase platform. It allows for raw execution of PostgreSQL-compatible statements.

Request Object Specification

FIELD	TYPE	REQUIRED	DESCRIPTION
`projectId`	String	Yes*	Target project ULID. Auto-injected if key is scoped.
`query`	String	Yes	The SQL statement to execute.
`params`	Array	No	List of values for parameterized queries (e.g. `$1`, `$2`).

Standard SELECT Example

```
JSON Request
{
  "projectId": "...",
  "query": "SELECT id, name, created_at FROM users WHERE email ILIKE $1",
  "params": ["%@gmail.com"]
}
```

Using parameters (`$1`) is strictly required for any user-inputted data to prevent SQL Injection attacks. Fluxbase uses prepared statements internally.

Standardized API Response

Every request returns a consistent wrapper object. Logic should always check the `success` boolean first.

```
JSON SUCCESS
{
  "success": true,
  "result": {
    "rows": [
      { "id": 1, "name": "Alice", "email": "alice@flux.com" }
    ],
    "columns": ["id", "name", "email"],
    "message": "Found 1 record"
  },
  "explanation": "Query used Index Scan on 'users_pkey'",
  "executionInfo": {
    "time": "4ms",
    "rowCount": 1
  }
}
```

The "Explanation" Field

In production environments, Fluxbase automatically appends query performance insights. If your query is performing slowly, this field will provide the `EXPLAIN ANALYZE` output from the underlying Postgres engine.

04. Complex Data Operations

Fluxbase supports complex SQL features including joins, aggregations, and subqueries.

Relational Joins

Retrieve data across multiple tables in a single network round-trip.

```
SQL
SELECT
  o.order_id,
  c.customer_name,
  o.total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.id
WHERE o.status = 'shipped'
ORDER BY o.created_at DESC;
```

Aggregations & Grouping

Perfect for generating analytics and dashboard summaries.

```
SQL
SELECT
  category,
  COUNT(*) as product_count,
  AVG(price) as average_price
FROM products
GROUP BY category
HAVING COUNT(*) > 5;
```

Note: Aliased columns (e.g. `COUNT(*) as product_count`) will appear in the `result.rows` array with those exact keys.

Database Migrations (DDL)

You can evolve your schema programmatically. Structural changes trigger a `schema_update` event for all active real-time listeners.

Creating Tables

```
SQL
CREATE TABLE inventory (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  sku TEXT UNIQUE NOT NULL,
  quantity INTEGER DEFAULT 0,
  last_restock TIMESTAMP WITH TIME ZONE
);
```

Altering Existing Schema

```
SQL
ALTER TABLE users ADD COLUMN phone_number VARCHAR(20);
ALTER TABLE users DROP COLUMN old_metadata_field;
```

Warning: Destructive operations (DROP TABLE, TRUNCATE) skip the Trash bin. Always back up your project before running structural deletions via the API.

05. Node.js Comprehensive Guide

Node.js is the first-class citizen in the Fluxbase ecosystem. We provide both a native REST approach and an official SDK.

Step 1: Environment Setup

Securely store your credentials in an `.env` file. Do not hardcode these in your primary script.

```
ENV
FLUXBASE_API_KEY=fl_live_abc123...
FLUXBASE_PROJECT_ID=proj_xyz789...
```

Step 2: Implementation (Native Fetch)

Use Node 18+ built-in `fetch` to minimize dependencies.

```
JAVASCRIPT
const runQuery = async (sql, params = []) => {
  const response = await fetch('https://fluxbase.vercel.app/api/execute-sql', {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${process.env.FLUXBASE_API_KEY}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      projectId: process.env.FLUXBASE_PROJECT_ID,
      query: sql,
      params: params
    })
  });

  const body = await response.json();
  if (!body.success) throw new Error(body.error.message);
  return body.result.rows;
};
```

Node.js: Practical Patterns

Handling mutations and stream selection in a production environment.

Writing Data with Parameters

```
JAVASCRIPT
const createTicket = async (title, content) => {
  const sql = `INSERT INTO tickets (title, body) VALUES ($1, $2) RETURNING *`;
  const [ticket] = await runQuery(sql, [title, content]);
  console.log(`Created ticket #${ticket.id}`);
  return ticket;
};
```

Batch Operations & Error Handling

```
JAVASCRIPT
try {
  const users = await runQuery('SELECT * FROM users');
  processUsers(users);
} catch (err) {
  if (err.message.includes('suspended')) {
    notifyAdmin('Organization is locked!');
  } else {
    console.error('Database Error:', err);
  }
}
```

NEXT.JS INTEGRATION

In Server Components, call the Fluxbase API directly within `async/await` functions. Fluxbase's 15s cache works seamlessly with Next.js's data-masking layers.

06. Python Integration Guide

The Python integration uses the popular `requests` library for robust HTTP communication.

Installation

`pip install requests python-dotenv`

Reference Library Implementation

```
PYTHON
import os
import requests
from dotenv import load_dotenv

load_dotenv()

class FluxbaseClient:
    def __init__(self):
        self.url = "https://fluxbase.vercel.app/api/execute-sql"
        self.key = os.getenv("FLUXBASE_API_KEY")
        self.pid = os.getenv("FLUXBASE_PROJECT_ID")

    def execute(self, sql, params=None):
        headers = {
            "Authorization": f"Bearer {self.key}",
            "Content-Type": "application/json"
        }
        payload = {
            "projectId": self.pid,
            "query": sql,
            "params": params or []
        }
        resp = requests.post(self.url, json=payload, headers=headers)
        data = resp.json()

        if not data["success"]:
            raise Exception(f"Fluxbase error: {data['error']['message']}")

        return data["result"]["rows"]
```

Python: Advanced Usage

Combining Fluxbase data with data science libraries like Pandas.

Loading into Pandas

```
PYTHON
import pandas as pd

client = FluxbaseClient()
rows = client.execute("SELECT * FROM sales_data")

# Create DataFrame directly from JSON rows
df = pd.DataFrame(rows)

print(df.describe())
print(f"Total Revenue: {df['amount'].sum()}")
```

FastAPI Service Integration

```
PYTHON
@app.get("/items")
def get_items(category: str):
    sql = "SELECT * FROM items WHERE category = $1"
    return client.execute(sql, [category])
```

Note: Fluxbase returns ISO-8601 strings for date/time columns. You can use `pd.to_datetime()` in Python to convert them for analysis.

07. Go Language (Reliable REST)

Go's strongly typed nature makes it an excellent choice for managing high-volume data pipelines with Fluxbase.

Infrastructure Setup

```
GO
type FluxRequest struct {
    ProjectID string   `json:"projectId"`
    Query     string   `json:"query"`
    Params    []interface{} `json:"params"`
}

type FluxResponse struct {
    Success bool `json:"success"`
    Result  struct {
        Rows []map[string]interface{} `json:"rows"`
    } `json:"result"`
    Error struct {
        Message string `json:"message"`
    } `json:"error"`
}
```

Execution Handler

Handling typed JSON in Go requires careful unmarshaling. We recommend a generic `[]map[string]interface{}` for row data.

Go: Production Client

```
GO
func RunQuery(sql string, params []interface{}) ([]map[string]interface{}, error) {
    client := &http.Client{Timeout: 10 * time.Second}
    payload := FluxRequest{
        ProjectID: os.Getenv("FLUX_PID"),
        Query:     sql,
        Params:    params,
    }

    body, _ := json.Marshal(payload)
    req, _ := http.NewRequest("POST", endpoint, bytes.NewBuffer(body))
    req.Header.Set("Authorization", "Bearer " + os.Getenv("FLUX_KEY"))
    req.Header.Set("Content-Type", "application/json")

    resp, err := client.Do(req)
    if err != nil { return nil, err }
    defer resp.Body.Close()

    var res FluxResponse
    json.NewDecoder(resp.Body).Decode(&res)

    if !res.Success { return nil, errors.New(res.Error.Message) }
    return res.Result.Rows, nil
}
```

Concurrency: Go's `goroutines` are massive advantages. You can fire multiple Fluxbase queries in parallel, but be mindful of the 30 req / 10 sec rate limit per project.

08. Rust (Performance & Safety)

Rust's zero-cost abstractions pair perfectly with Fluxbase. Use `reqwest` and `serde_json` for a modern type-safe setup.

Cargo Config

```
TOML
[dependencies]
reqwest = { version = "0.12", features = ["json"] }
tokio = { version = "1.0", features = ["full"] }
serde_json = "1.0"
```

Client Implementation

```
RUST
use serde_json::{json, Value};

async fn flux_query(sql: &str) -> Result> {
    let client = reqwest::Client::new();
    let body = json!({
        "projectId": std::env::var("FLUX_PID")?,
        "query": sql
    });

    let resp = client.post("https://fluxbase.vercel.app/api/execute-sql")
        .header("Authorization", format!("Bearer {}", std::env::var("FLUX_KEY")?))
        .json(&body)
        .send().await?
        .json::().await?;

    Ok(resp)
}
```

Rust: Type-Safe Rows

Map JSON results directly to your defined structs using Serde.

```
RUST
#[derive(Deserialize, Debug)]
struct User {
    id: i32,
    username: String,
    created_at: String,
}

#[tokio::main]
async fn main() {
    let raw = flux_query("SELECT * FROM users").await.unwrap();
    let users: Vec = serde_json::from_value(raw["result"]["rows"].clone()).unwrap();

    for u in users {
        println!("User: {} (ID: {})", u.username, u.id);
    }
}
```

Error Handling: Rust's `Result` type should be used to catch connection timeouts and Fluxbase suspension errors before they reach your logic layer.

09. Java (Enterprise Ready)

The Java SDK approach uses the standard `HttpClient` (Java 11+) for a zero-dependency integration profile.

The FluxClient Class

```
JAVA
public class FluxClient {
    private static final String URL = "https://fluxbase.vercel.app/api/execute-sql";
    private final String apiKey;
    private final String projectId;

    public FluxClient(String apiKey, String projectId) {
        this.apiKey = apiKey;
        this.projectId = projectId;
    }

    public String execute(String sql) throws Exception {
        HttpClient client = HttpClient.newHttpClient();
        String json = String.format("{\"projectId\":\"%s\", \"query\":\"%s\"}", projectId, sql);

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(URL))
            .header("Authorization", "Bearer " + apiKey)
            .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(json))
                .build();

        return client.send(request, HttpResponse.BodyHandlers.ofString()).body();
    }
}
```

Java: Spring Boot Context

Spring Boot developers can utilize `RestTemplate` or `WebClient` for reactive streaming.

```
JAVA
@Service
public class DataService {
    @Autowired private RestTemplate restTemplate;

    public List getAllUsers() {
        HttpHeaders headers = new HttpHeaders();
        headers.setBearerAuth(env.getProperty("flux.apiKey"));

        Map body = new HashMap<>();
        body.put("projectId", env.getProperty("flux.pid"));
        body.put("query", "SELECT * FROM users");

        HttpEntity entity = new HttpEntity<>(body, headers);
        FluxResponse res = restTemplate.postForObject(URL, entity, FluxResponse.class);

        return res.getResult().getRows();
    }
}
```

10. Event-Driven Webhooks

Webhooks allow Fluxbase to notify your application of changes automatically. This eliminates the need for polling and reduces costs.

HOW IT WORKS

1. An event occurs (e.g. `row.inserted`) in your table.
2. Fluxbase generates a secure JSON payload.
3. Fluxbase fires an outbound `POST` to your registered URL.
4. Your app responds with `200 OK` to acknowledge receipt.

Event Types

- `row.inserted`: Fired after a successful `INSERT` or Bulk Upload.
- `row.updated`: Fired for every record modified by an `UPDATE` statement.
- `row.deleted`: Fired when a row matches a `DELETE` criteria.
- `*`: Wildcard event listener (includes all above).

Payload Structure

```JSON
{
  "event_type": "row.inserted",
  "table_id": "orders",
  "timestamp": "2026-04-02T...",
  "data": {
    "new": { "id": "ORD55", "total": 99.99 },
    "old": null
  }
}
```

Deploying Webhook Receivers

Receivers must be reachable over the public internet. We recommend using serverless functions for scalability.

Express (Node.js) Implementation

```
JAVASCRIPT
app.post('/webhooks/fluxbase', (req, res) => {
  const { event_type, table_id, data } = req.body;

  if (table_id === 'orders') {
    handleOrderUpdate(data.new);
  }

  res.sendStatus(200); // Standard acknowledgement
});
```

Verification & Security

To ensure requests are genuinely from Fluxbase, check the `X-Fluxbase-Signature` header. This is a HMAC SHA256 hash of the payload using your Webhook Secret.

```
JAVASCRIPT (PSEUDO)
const signature = req.headers['x-fluxbase-signature'];
const isValid = verifyHmac(req.body, secret, signature);
if (!isValid) return res.status(401);
```

11. Storage: S3-Backed Logic

Fluxbase Storage provides a managed interface for binary assets. All files are encrypted at rest on AWS S3.

The Lifecycle of an Asset

1. **Create Bucket:** Organize files logically (e.g. "avatars", "invoices").
2. **Upload:** Send `multipart/form-data` to `/api/storage/upload`.
3. **Index:** Store the `s3_key` in your primary SQL table.
4. **Serve:** Generate a short-lived (15m) URL to display the asset.

Buckets Architecture

Buckets can be Private (default) or Public. Public buckets generate stable URLs (S3 origin), while Private buckets require an API Key to generate a signed access token.

```
JSON (Create Bucket)
{
  "projectId": "...",
  "name": "project-assets",
  "isPublic": false
}
```

Managing Files

The storage API handles complex S3 multipart uploads automatically. You simply provide the file object.

Upload Implementation (JavaScript)

```
JAVASCRIPT
const uploadFile = async (file, bucketName) => {
  const formData = new FormData();
  formData.append('file', file);
  formData.append('bucketId', bucketName);
  formData.append('projectId', PID);

  const res = await fetch('/api/storage/upload', {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${KEY}` },
    body: formData
  });

  return await res.json();
};
```

Generating Download URLs

Because files are private, you must "hand-shake" to get a viewable URL.

```
HTTP GET
/api/storage/url?s3Key=project_1/avatars/user_1.jpg&projectId=proj_1
```

Expires: 900 seconds (15 minutes). After expiration, the link returns 403 Access Denied from S3.

12. Real-time (SSE) Subscriptions

Fluxbase uses Server-Sent Events (SSE) to push live updates. Unlike WebSockets, SSE is unidirectional (Server → Client), making it lighter and better for browser performance.

The Real-time Sidecar

Because Fluxbase APIs run on Serverless headers (Vercel), we maintain persistent **Render Sidecars** to keep connections alive.

> **Config Checklist:** You need both a `projectId` and a `realtimeUrl`. The default realtime URL for v4.1 is:
> `https://fluxbase-live.onrender.com`

JavaScript SDK Initialization

```
TYPESCRIPT
import { createClient } from '@fluxbaseteam/fluxbase';

const flux = createClient(URL, PID, KEY, {
  realtimeUrl: 'https://fluxbase-live.onrender.com'
});
```

Working with Channels

Channels allow you to group subscriptions by table or logical "rooms".

Listening for Row Inserts

```
TYPESCRIPT
const channel = flux.channel('messages_feed', 'messages')
  .on('row.inserted', (payload) => {
    updateUI(payload.data.new);
  })
  .subscribe();

// To stop listening:
channel.unsubscribe();
```

The Heartbeat Mechanism

Fluxbase emits a `:keepalive` heartbeat every 15 seconds. The SDK automatically detects drops and initiates an **Exponential Backoff** reconnection strategy (1s → 2s → 4s → 8s... max 30s).

Tip: Call `channel.pause()` when the user minimizes the tab to save bandwidth, and `channel.resume()` on visibility change.

13. Case Study: Live Chat App

In this walkthrough, we will build a production-ready anonymous chat application using Fluxbase SQL and Real-time SSE.

The Schema

We need a single table to hold our messages. We'll use the SQL editor to create it:

```
SQL
CREATE TABLE messages (
  id SERIAL PRIMARY KEY,
  sender_name TEXT,
  content TEXT NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

Step 1: Sending Messages

When the user hits "Send", we fire a mutation via the API.

```
JAVASCRIPT
const sendMessage = async (name, txt) => {
  await flux.execute(
    'INSERT INTO messages (sender_name, content) VALUES ($1, $2)',
    [name, txt]
  );
};
```

Live Chat: UI Reactive Sync

To make the chat "real-time", we don't refresh the page. We listen for the `row.inserted` event.

Step 2: Monitoring the Channel

```
JAVASCRIPT
const setupLiveChat = (setMessages) => {
  return flux.channel('main_chat', 'messages')
    .on('row.inserted', (payload) => {
      // Direct state update from the wire
      setMessages(prev => [...prev, payload.data.new]);
    })
    .subscribe();
};
```

The Result

With less than 20 lines of application code, we've built a system that supports thousands of concurrent users with sub-100ms message delivery across continents.

Deployment Note: Host the frontend on Vercel and the background processing on Render. This follows the Fluxbase **Dual-Route architecture** for maximum stability.

Live Chat: Security & Hardening

To prevent spam and unauthorized data leaks, we apply **Row Level Security (RLS)** and API Rate limits.

1. Rate Limiting

Fluxbase enforces a 429 limit. For a chat app, we should also implement a debouncer on the send button to prevent accidental floods.

2. Sanitization

While Fluxbase prevents SQL injection, your app should still sanitize the **HTML output** when rendering messages to prevent XSS attacks.

```
JAVASCRIPT (React)
// Use browser native escape or a library
const cleanContent = escape(message.content);
return
{cleanContent}
;
```

3. Audit Logs

Enable Audit Logging in the Fluxbase dashboard. This tracks every mutation event, giving you a clear history of which API Key sent which message, crucial for moderation.

14. Organization Governance

New in v4.1, the **Suspension Engine** provides a safety switch for administrators and developers to immediately "pause" infrastructure usage.

The "Suspended" State

When an organization is suspended, Fluxbase places a virtual lock on the project schema. No SQL—read or write—is processed until the status is cleared.

EXECUTION BLOCKED

API responses will trigger a **403 FORBIDDEN** with the specific message: `"Organization suspended. Please resume in Settings."`

Impact on Background Services

- **Webhook Pipeline:** All event notifications are discarded. No HTTP requests are fired to your servers.
- **SSE Streams:** Current connections are forcefully terminated. Subsequent attempts to re-subscribe will fail during the handshake.
- **Storage:** File uploads are blocked, though existing public URLs remain valid until their natural expiry.

Resuming Operations

Restoring access is instant. Once an administrator clicks "Resume Organization" in the Settings portal, the lock is lifted globally within <500ms.

15. Error Reference Guide

Standardized numeric and string codes returned by all API layers.

ERROR CODE	HTTP	RESOLUTION
`AUTH_REQUIRED`	401	Check `Authorization` Bearer token.
`SCOPE_MISMATCH`	403	Key not allowed for this `projectId`.
`FORBIDDEN`	403	Organization suspended or insufficient role.
`RATE_LIMIT_EXCEEDED`	429	Wait 10s and retry with exponential backoff.
`PROJECT_NOT_FOUND`	404	Verify project ID in Dashboard URL.
`SQL_EXEC_ERROR`	200	Check query syntax in the SQL editor.
`INTERNAL_ERROR`	500	Platform issue — contact support.

HTTP 200 vs 500

Fluxbase returns `200 OK` for syntax errors (with `success: false`) to prevent standard REST clients from failing silently. Always check the `success` property in your JSON response.

Usage Quotas & Service Limits

Fluxbase operates with the following default soft-limits to ensure platform stability. Higher limits are available for Enterprise customers.

QUERY LIMITS

- Concurrent Queries: 50
- Max Rows (Read): 10,000
- Query Timeout: 60s

STORAGE LIMITS

- Max File Size: 2 GB
- Bandwidth / mo: Unlimited
- Buckets / Project: 100

System Status

Monitor real-time platform health at: `https://status.fluxbase.app`

FLUXBASE

Thank you for integrating with Fluxbase.
We are building the future of the agentic web together.